Wednesday 12:19 AM

> Olá, vamos continuar editando meu gerador de músicas com GA, ok?

Olá! Claro. Vamos continuar do ponto em que paramos no gerador de músicas com **Algoritmo Genético (GA)**.

Pode me enviar o **código atual** ou dizer qual é o próximo ajuste que você quer fazer. Vou manter a estrutura que já definimos (MChords, bateria, Guitar, Strings etc.) e evitar adicionar complexidade desnecessária.

> Os MChords estão ficando com acordes repetidos, mas quero garantir que isso não aconteça, mexendo na fitness. Lembrando que são 8 MChords. Vou te passar o código da fitness e você edita ela para valorizar muito mais os indivíduos com acordes distintos, ok?

Perfeito. Envie o código atual da `fitness.py`.

Vou alterar **somente a parte necessária da fitness** para que, entre os **8 MChords**, indivíduos com acordes distintos recebam uma recompensa **bem maior**, penalizando fortemente repetições. A ideia será fazer a diversidade dos 8 acordes pesar bastante no fitness, sem destruir os outros critérios musicais que você já possui.

Sim. O problema principal está justamente na `harmonic_diversity_fitness()` atual: ela usa uma **gaussiana com alvo de apenas 50% de raízes distintas**, então ela pode considerar perfeitamente aceitável que vários dos 8 MChords sejam repetidos.

Como agora temos **exatamente 8 MChords**, eu mudaria a lógica para:

- **8 acordes diferentes → pontuação máxima**
- 7 diferentes → muito alta
- 6 → alta
- 5 → razoável
- 4 ou menos → penalização forte
- além disso, considerar um acorde como repetido quando **root + chord_type + inversion** forem iguais.
- aumentar também o peso da diversidade dentro da harmonia.

1. Substitua `harmonic_diversity_fitness()` inteira por esta

Python

```
def harmonic_diversity_fitness(individual):
    """
    Incentiva fortemente a diversidade dos 8 MChords.

    Um acorde é considerado igual a outro quando possui a mesma:
        - root
        - chord_type
        - inversion

    Como existem exatamente 8 MChords, a função recompensa
    progressivamente indivíduos com mais acordes distintos.

    Objetivo principal:
        8 acordes distintos -> 1.00
        7 acordes distintos -> pontuação muito alta
        6 acordes distintos -> alta
        5 acordes distintos -> moderada
        4 ou menos -> penalização forte
    """

    chords = get_mchords(individual)
```

2. E aumente o peso dela em `harmony_details()`

Atualmente você tem:

Python

```
harmony = (
    0.10 * chord_validity
    + 0.15 * tonal_coherence
    + 0.15 * chord_quality
    + 0.10 * root_motion
    + 0.10 * voice_leading
    + 0.15 * base_stability
    + 0.15 * chorus_score
    + 0.10 * harmonic_diversity
)
```

Eu mudaria para:

Python

```
harmony = (
    0.10 * chord_validity
    + 0.15 * tonal_coherence
    + 0.15 * chord_quality
    + 0.10 * root_motion
    + 0.10 * voice_leading
    + 0.10 * base_stability
    + 0.10 * chorus_score
    + 0.20 * harmonic_diversity
)
```

Assim, a diversidade passa de **10% para 20% da fitness de harmonia**, enquanto mantemos os demais critérios.

Por que essa versão é melhor para o seu caso

A versão anterior fazia algo como:

> "Não tem problema ter poucas raízes distintas; quero algo em torno de metade."

Isso é justamente contrário ao que você quer agora.

A nova versão pergunta diretamente:

> **Dos 8 MChords, quantos são realmente diferentes?**

Por exemplo:

MChords	Distintos	Resultado
A B C D E F G H	8/8	⭐⭐⭐⭐⭐
A B C D E F G A	7/8	⭐⭐⭐⭐
A B C D E F A A	6/8	⭐⭐⭐
A B C D A B C D	4/8	⭐⭐
A A B B A A B B	2/8	⭐
A A A A A A A A	1/8	×

E tem uma consequência importante para o GA: **um indivíduo com boa harmonia, mas que repete muito os MChords, agora perde bastante fitness**, dando pressão evolutiva para que

mutação/crossover produzam progressões mais variadas.

Eu manteria **root + chord_type + inversion** como identidade do acorde por enquanto, porque isso evita considerar duas inversões diferentes como literalmente o mesmo MChord.

Não entendi a parte de substituir o harmonic_detailes...

Claro. Eu me expressei de forma confusa.

Você **não precisa substituir `harmony_details()` inteira**. O que eu quis dizer é apenas alterar **o peso que a `harmonic_diversity` recebe dentro dela**.

Se no seu código está assim:

```python
harmony = (
    0.10 * chord_validity
    + 0.15 * tonal_coherence
    + 0.15 * chord_quality
    + 0.10 * root_motion
    + 0.10 * voice_leading
    + 0.15 * base_stability
    + 0.15 * chorus_score
    + 0.10 * harmonic_diversity
)
```

troque **somente essa parte** por:

```python
harmony = (
    0.10 * chord_validity
    + 0.15 * tonal_coherence
    + 0.15 * chord_quality
    + 0.10 * root_motion
    + 0.10 * voice_leading
    + 0.10 * base_stability
    + 0.10 * chorus_score
    + 0.20 * harmonic_diversity
)
```

Ou seja, fizemos duas coisas:

Plain text

```
base_stability:     0.15 → 0.10
chorus_score:       0.15 → 0.10
harmonic_diversity: 0.10 → 0.20
```

O objetivo é simplesmente **dar o dobro da importância para a diversidade dos MChords**.

Mas há uma coisa importante

Se você quer que a fitness **valorize MUITO** os 8 MChords diferentes, eu acho melhor não mexermos ainda nos outros pesos.

Podemos fazer algo mais direto:

Python

```
harmony = (
    0.10 * chord_validity
    + 0.15 * tonal_coherence
    + 0.15 * chord_quality
    + 0.10 * root_motion
    + 0.10 * voice_leading
    + 0.15 * base_stability
    + 0.15 * chorus_score
    + 0.10 * harmonic_diversity
)
```

e modificar **apenas a função `harmonic_diversity_fitness()`** para que ela seja muito mais agressiva contra acordes repetidos.

Nesse caso, **nenhuma outra parte da sua fitness é alterada**.

Para o que você pediu, eu prefiro essa segunda abordagem.

> Ok, fiz aqui e melhorou mesmo, agora com relação à bateria, preciso reduzir a aparição do item CRASH, tanto no Batt1 quanto no Batt2, mas quero fazer isso mexendo só na fitness se possível. Já que você viu o código dela, me diga como resolver.

Sim. Como você quer **mexer somente na fitness**, eu faria isso adicionando uma penalização específica para a frequência de `CRASH` em `Bat1` e `Bat2`.

A ideia é **não proibir o CRASH**, apenas fazer o GA preferir indivíduos onde ele aparece menos.

Como já vi a estrutura da sua `fitness.py`, o ideal é colocar essa penalização junto aos critérios de bateria, em vez de alterar o gerador dos genes.

Como eu faria

Para cada bateria:

- contar quantos `CRASH` existem;
- calcular a proporção de CRASH em relação aos elementos da bateria;
- quanto maior essa proporção, menor a pontuação;
- aplicar uma penalização relativamente forte.

Por exemplo, podemos criar:

```python
def crash_frequency_fitness(individual):
    """
    Penaliza a presença excessiva de CRASH em Bat1 e Bat2.
    """

    bats = [
        individual.Bat1,
        individual.Bat2
    ]

    total_items = 0
    total_crashes = 0

    for bat in bats:
        for item in bat:
            total_items += 1

            if item == CRASH:
                total_crashes += 1

    if total_items == 0:
```

Depois você adicionaria essa função à fitness da bateria.

Mas há um detalhe importante: preciso respeitar exatamente como `Bat1` e `Bat2` estão representados no seu código atual. Por exemplo, se cada bateria é uma lista de genes, uma matriz, ou uma sequência de índices, o acesso ao `CRASH` muda.

Como você me passou a `fitness.py` anteriormente, posso editar **em cima da estrutura exata dela**. Se quiser, eu posso te devolver agora **a `fitness.py` completa já modificada**, mantendo tudo que

você acabou de ajustar nos MChords e adicionando apenas a penalização do CRASH.

O código de fitness que te passei não olha os detalhes da bateria pra defini-la ?

Worked for 6s

Sim — e você está certo em questionar isso. Olhando novamente para o código que você enviou, **a fitness já analisa os detalhes internos da bateria**.

Por exemplo, ela já verifica coisas como:

- quantidade de eventos;
- densidade;
- posições dos elementos;
- presença de `SNARE` no backbeat;
- atividade dos steps;
- padrão do kick;
- regularidade de hi-hat;
- síncope;
- combinações entre instrumentos;
- características específicas de `Bat1` e `Bat2`.

Inclusive, no `chorus_bat2_fitness()`, ela acessa diretamente:

```python
if SNARE in gene.notes
```

Ou seja, cada `gene` possui seus elementos de bateria e a fitness consegue saber **quais elementos estão presentes em cada step**. Pasted markdown(1)

E o próprio arquivo deixa claro que os elementos são:

```
0 = Kick
1 = Snare
2 = Closed Hi-Hat
3 = Open Hi-Hat
4 = Crash
```

Pasted markdown(1)

Então podemos fazer exatamente o que você quer

Não precisamos criar uma lógica genérica como a que sugeri antes.

Podemos criar uma pequena função, por exemplo:

```python
def crash_fitness(individual):
    ...
```

que percorra:

```python
Bat1 → gene.notes
Bat2 → gene.notes
```

e conte especificamente o elemento:

```python
CRASH
```

Depois incorporamos essa pontuação à **fitness da bateria**, sem modificar a geração dos genes.

Isso é inclusive melhor porque podemos definir algo musicalmente mais adequado, por exemplo:

- **0–1 Crash** → excelente
- **2 Crash** → bom
- **3–4 Crash** → aceitável
- **5+ Crash** → penalização forte

E não simplesmente dizer "quanto menos crash, melhor", porque **Crash não deve desaparecer completamente**. Ele pode ser útil para marcar transições, finais de frases ou momentos fortes.

Se você quiser, posso olhar **a função que atualmente combina os critérios de bateria** na sua `fitness.py` e te dizer exatamente **onde inserir essa penalização e com quais pesos**, sem alterar nenhuma outra parte.

Me diga o que eu devo mudar no código da fitness para forçar a redução do aparecimento de CRASH (mas não totalmente)

Sim. Como a sua `fitness.py` **já percorre os `gene.notes` da bateria**, eu faria uma alteração bem localizada: criar uma função que **penaliza o excesso de CRASH**, mas mantém uma pequena quantidade desejável.

1. Adicione esta função

Coloque junto das outras funções de avaliação da bateria:

Python

```python
def crash_fitness(individual):
    """
    Penaliza o excesso de CRASH em Bat1 e Bat2,
    mas não elimina completamente sua utilização.
    """

    bat1 = get_bat1(individual)
    bat2 = get_bat2(individual)

    total_crashes = 0
    total_steps = 0

    for bat in (bat1, bat2):
        if not bat:
            continue

        total_steps += len(bat)

        for gene in bat:
            total_crashes += gene.notes.count(CRASH)
```

2. Depois, encontre a função que calcula a fitness da bateria

Na sua `fitness.py`, você já possui vários componentes para avaliar a bateria. Adicione:

Python

```python
crash_score = crash_fitness(individual)
```

junto aos outros scores.

Depois inclua esse score na combinação da bateria. Por exemplo, se atualmente você tiver algo parecido com:

```python
battery = (
    0.20 * strong_beats
    + 0.20 * backbeat
    + 0.20 * kick_pattern
    + 0.20 * hihat_pattern
    + 0.20 * density
)
```

pode fazer:

```python
battery = (
    0.18 * strong_beats
    + 0.18 * backbeat
    + 0.18 * kick_pattern
    + 0.18 * hihat_pattern
    + 0.18 * density
    + 0.10 * crash_score
)
```

Assim, **10% da avaliação da bateria passa a depender diretamente da quantidade de CRASH**.

O comportamento será aproximadamente

Considerando 32 steps no total entre `Bat1` e `Bat2`:

CRASH	Situação
0	Muito bom
1–2	**Ideal**
3	Bom
4	Começa a perder fitness
5–6	Penalização forte

CRASH	Situação
7+	Penalização muito forte

A vantagem é que **não estamos proibindo CRASH**. Um indivíduo com 1 ou 2 CRASH pode ter uma fitness melhor que um indivíduo sem nenhum, dependendo dos demais critérios.

Importante: eu não mexeria na geração da bateria nem removeria `CRASH` dos genes. O GA continuará podendo gerar CRASH livremente; a diferença é que agora a seleção tenderá a eliminar os indivíduos que abusam dele.

Se você quiser uma redução **ainda mais forte**, o primeiro parâmetro que eu aumentaria seria o peso de `crash_score` de `0.10` para `0.15`, sem precisar alterar a função.

> Tá meio complicado isso, vamos tentar algo mais simples, qual outor elemento eu poderia colocar no CRASH: "Crash", para troca-lo por um item menos intenso?

Sim. Se a ideia é **simplesmente substituir o `CRASH` por algo menos intenso**, eu usaria:

- `Closed Hi-Hat` → **melhor opção** se você quer algo mais discreto e frequente.
- `Open Hi-Hat` → um pouco mais destacado, mas ainda bem menos intenso que Crash.
- `Snare` → mantém bastante presença rítmica, mas não tem o caráter de “acento” do Crash.

No seu caso, eu escolheria **`Closed Hi-Hat`**.

Então, onde você tem:

```python
CRASH
```

poderia simplesmente trocar por:

```python
CLOSED_HIHAT
```

ou pelo nome constante que você estiver usando para o índice `2`.

Musicalmente:

`Crash` = acento forte / transição / início de seção

`Open Hi-Hat` = acento moderado

`Closed Hi-Hat` = elemento rítmico leve e contínuo

Se você quer **diminuir o aparecimento do Crash sem alterar a estrutura do GA**, essa é provavelmente a solução mais simples.

Já to usando o CLOSEDHIHAT em outro, ficaria duplicado

Sim, nesse caso eu evitaria duplicar o `CLOSED_HIHAT`.

Eu colocaria **`OPEN_HIHAT`** no lugar do `CRASH`.

A hierarquia ficaria bem natural:

```
Plain text

Kick          → grave
Snare         → médio/forte
Closed Hi-Hat → leve
Open Hi-Hat   → médio, mais destacado
Crash         → muito forte
```

Então, se o objetivo é **reduzir a intensidade do Crash sem criar duplicação**, a troca mais lógica é:

```
Python

CRASH → OPEN_HIHAT
```

O `OPEN_HIHAT` ainda dá um **acento** ao padrão, mas é consideravelmente menos agressivo que um Crash.

```
DRUM_PITCHES = {
  KICK: 36,
  SNARE: 38,
  CLOSED_HIHAT: 42,
  OPEN_HIHAT: 46,
  CRASH: 49,
}
```

Mudar esse PITCHE do CRASH para outra coisa parecida sem ser os que ja estao ai, é possivel?

Sim, é possível. O `CRASH` não precisa necessariamente usar uma das outras notas que você já definiu.

Na bateria General MIDI, existem vários instrumentos entre esses valores. Por exemplo:

```python
DRUM_PITCHES = {
    KICK: 36,
    SNARE: 38,
    CLOSED_HIHAT: 42,
    OPEN_HIHAT: 46,
    CRASH: 49,
}
```

Você poderia trocar o `49` por:

```python
CRASH: 51,  # Ride Cymbal 1
```

O **Ride Cymbal** é provavelmente a melhor alternativa para o que você quer: é uma percussão de prato, portanto mantém uma função semelhante ao Crash, mas tem um caráter **menos explosivo e mais contínuo**.

Outras opções:

MIDI	Instrumento	Intensidade
49	Crash Cymbal 1	🔴 Forte
51	**Ride Cymbal 1**	🟡 Moderada
53	Ride Bell	🟡 Moderada/forte
57	Crash Cymbal 2	🔴 Forte
59	Ride Cymbal 2	🟡 Moderada

Eu usaria:

Python

```
CRASH: 51
```

Assim você **mantém o elemento chamado `CRASH` no código/GA**, mas sonoramente ele passa a ser um **Ride Cymbal**, sem duplicar Kick, Snare, Closed Hi-Hat ou Open Hi-Hat.

Melhorou

Ótimo! 👍

Então podemos manter:

Python

```
CRASH: 51,  # Ride Cymbal 1
```

Isso preserva toda a lógica do seu GA — o gene continua sendo chamado `CRASH` — mas o som gerado fica **bem menos agressivo** e ainda funciona como um elemento de prato/acento.

Acho que essa solução é melhor do que mexer na fitness.